

Documentation projet morphing en C :

Guide de compilation :

Avant de compiler, vous devez installer ces dépendances :

ImageMagick - Conversion et redimensionnement d'images

- Utilisé pour : Convertir vos images JPG/PNG en format PPM
- macOS : `brew install imagemagick`
- Linux : `sudo apt install imagemagick`
- Windows : Télécharger sur <https://imagemagick.org>

SDL2 + extensions - Bibliothèques graphiques

- Utilisé pour : Interface de sélection des points (fenêtre, dessin, événements souris)
- macOS : `brew install sdl2 sdl2_ttf sdl2_gfx`
- Linux : `sudo apt install libsdl2-dev libsdl2-ttf-dev libsdl2-gfx-dev`
- Windows : Télécharger sur <https://www.libsdl.org>

FFmpeg - Création de vidéos

- Utilisé pour : Assembler les images générées en vidéo MP4
- macOS : `brew install ffmpeg`
- Linux : `sudo apt install ffmpeg`
- Windows : Télécharger sur <https://ffmpeg.org>

Outils de compilation (si absents)

- macOS : Installer Xcode Command Line Tools avec `xcode-select --install`
 - Linux : `sudo apt install build-essential`
 - Windows : Installer MinGW-w64 ou utiliser WSL2 (recommandé)
-

Vérification des installations :

bash

`magick --version`

`ffmpeg -version`

18:37

5/5

Commandes de compilation :

Compilation automatique (avec Makefile) :

bash

Nettoyer les anciens fichiers

make clean

Compiler le programme

make morphing

Compilation manuelle :

1. Créer le dossier pour les fichiers objets :

bash

mkdir -p obj

2. Compiler chaque fichier source (.c) en fichier objet (.o) :

bash

Compiler image.c

```
gcc -Wall -Wextra -g -I./include -I./IN304_Projet \  
-I/opt/homebrew/opt/sdl2/include/SDL2 \  
-I/opt/homebrew/opt/sdl2_ttf/include/SDL2 \  
-c src/image.c -o obj/image.o
```

Compiler selection_points.c

```
gcc -Wall -Wextra -g -I./include -I./IN304_Projet \  
-I/opt/homebrew/opt/sdl2/include/SDL2 \  
-I/opt/homebrew/opt/sdl2_ttf/include/SDL2 \  
-c src/selection_points.c -o obj/selection_points.o
```

Compiler morphing.c

```
gcc -Wall -Wextra -g -I./include -I./IN304_Projet \  
-I/opt/homebrew/opt/sdl2/include/SDL2 \  
-I/opt/homebrew/opt/sdl2_ttf/include/SDL2 \  
-c src/morphing.c -o obj/morphing.o
```

Compiler main_final.c

```
gcc -Wall -Wextra -g -I./include -I./IN304_Projet \  

```

```
-I/opt/homebrew/opt/sdl2/include/SDL2 \  
-I/opt/homebrew/opt/sdl2_ttf/include/SDL2 \  
-c src/main_final.c -o obj/main_final.o
```

Compiler uvsqgraphics_2.c

```
gcc -Wall -Wextra -g -I./include -I/IN304_Projet \  
-I/opt/homebrew/opt/sdl2/include/SDL2 \  
-I/opt/homebrew/opt/sdl2_ttf/include/SDL2 \  
-c IN304_Projet/uvsqgraphics_2.c -o obj/uvsqgraphics_2.o
```

3. Lier tous les fichiers objets pour créer l'exécutable :

bash

```
gcc -o morphing obj/main_final.o obj/image.o \  
obj/selection_points.o obj/morphing.o obj/uvsqgraphics_2.o \  
-L/opt/homebrew/opt/sdl2/lib \  
-L/opt/homebrew/opt/sdl2_ttf/lib \  
-L/opt/homebrew/opt/sdl2_gfx/lib \  
-lSDL2 -lSDL2_ttf -lSDL2_gfx -lm
```

Note : Sur Linux, remplacez `/opt/homebrew/` par `/usr/` dans tous les chemins.

Après la compilation, voici les étapes à suivre :

Étape 1 : Préparer vos images

Convertir vos images au format PPM et les redimensionner :

bash

```
# Convertir une image JPG/PNG en PPM avec une taille de 400x400  
magick chat.jpg -resize 400x400 images_ppm/chat.ppm  
magick chien.jpg -resize 400x400 images_ppm/chien.ppm
```

Recommandations de taille :

- 200×200 : Tests rapides
 - 400×400 : Qualité correcte (recommandé)
 - 600×600 : Meilleure qualité (plus lent)
-

Étape 2 : Lancer le programme

bash

```
./morphing images_ppm/chat.ppm images_ppm/chien.ppm 30
```

Où :

- `images_ppm/chat.ppm` = image de départ
 - `images_ppm/chien.ppm` = image d'arrivée
 - 30 = nombre d'images intermédiaires (génère 31 images au total)
-

Étape 3 : Sélectionner les points dans l'interface

Une fenêtre s'ouvre automatiquement :

1. Cliquez sur un point dans l'image de GAUCHE
 2. Cliquez sur le point correspondant dans l'image de DROITE
 3. Répétez pour 5-10 couples de points
 4. Cliquez sur le bouton **SAUVER** (cyan)
 5. Cliquez sur le bouton **QUITTER** (rouge)
-

Étape 4 : Visualiser la vidéo

bash

macOS

```
open morphing.mp4
```

Linux

```
xdg-open morphing.mp4
```

Windows

```
start morphing.mp4
```

La vidéo dure environ 10 secondes et montre la transformation progressive.

Organisation du projet :

Afin de bien travailler, nous avons créé une structure de dossiers claire et organisée :

src/ - Code source

- Contient tous les fichiers .c du projet
- Regroupe les différents modules : gestion d'images, interface graphique, calculs de morphing, programme principal

include/ - Fichiers d'en-tête

- Contient les fichiers .h avec les déclarations de structures et prototypes de fonctions
- Permet de partager les définitions entre les différents fichiers sources

obj/ - Fichiers objets

- Stocke les fichiers .o générés lors de la compilation
- Fichiers intermédiaires entre le code source et l'exécutable final

images_origine/ - Images d'origine

- Conserve les images sources au format JPG, PNG ou autre
- Images avant conversion au format PPM

images_ppm/ - Images converties

- Contient les images converties au format PPM
- Format utilisé par le programme pour le morphing

images_out/ - Images générées

- Accueille les images intermédiaires créées par le programme
- Ces images sont ensuite assemblées pour créer la vidéo finale

data/ - Données complémentaires

- Stocke les fichiers de couples de points sélectionnés par l'utilisateur
- Format texte contenant les coordonnées des points de correspondance entre les deux images

Partie 2 : Lecture et affichage des images

Ressources consultées

Pour implémenter cette partie, nous avons consulté les ressources suivantes :

- Documentation du format PPM : https://fr.wikipedia.org/wiki/Portable_pixmap
- Spécifications techniques PPM : <https://netpbm.sourceforge.net/doc/ppm.html>
- Documentation ImageMagick : <https://imagemagick.org>

Notre implémentation supporte à la fois le format **P3** (ASCII, lisible) et le format **P6** (binaire, compact) pour la lecture des fichiers PPM. Pour l'écriture, nous utilisons exclusivement le format **P6** afin d'optimiser la taille des fichiers générés.

2.1 - Compréhension d'ImageMagick

ImageMagick est un outil en ligne de commande permettant la manipulation d'images. Dans notre projet, il sert à deux opérations essentielles : la conversion d'images de formats standards (JPG, PNG) vers le format PPM requis par notre programme, et le redimensionnement des images à une taille optimale pour le traitement.

2.2 - Organisation des dossiers

Nous avons créé deux dossiers distincts pour séparer clairement les différentes étapes du traitement :

- **images_origine/** : Répertoire contenant les images sources dans leur format original (JPG, PNG, etc.) avant toute transformation.
- **images_ppm/** : Répertoire destiné à accueillir les images converties au format PPM, prêtes à être utilisées par le programme de morphing.

Cette séparation permet de conserver les fichiers originaux intacts et de faciliter la gestion des différentes versions des images.

2.3 - Commandes ImageMagick définies

La commande principale utilisée pour la conversion est :

bash

```
magick input.jpg -resize 400x400 output.ppm
```

Cette commande effectue deux opérations simultanées : elle convertit l'image source au format PPM et la redimensionne à 400×400 pixels, une taille optimale pour garantir un bon équilibre entre qualité visuelle et performance de traitement.

2.4 - Programme de conversion en C

Fichier créé : `src/convertir_images.c`

Ce programme permet d'automatiser la conversion des images.

Fonction implémentée :

convertir_image() : Cette fonction prend en paramètres le chemin du fichier d'entrée, le chemin du fichier de sortie ainsi que les dimensions souhaitées. Elle construit dynamiquement la commande ImageMagick appropriée et l'exécute.

Fonction externe utilisée :

- **system()** (de `stdlib.h`) : Exécute la commande ImageMagick construite pour convertir l'image.

2.5 - Structures de données

Fichier créé : `include/image.h`

Ce fichier d'en-tête définit l'ensemble des structures de données utilisées dans le projet :

Structure Point : Représente un point dans le plan 2D avec ses coordonnées entières x et y. Cette structure est utilisée pour stocker les positions des pixels, des points de base et des sommets de triangles.

Structure Pixel : Encode une couleur au format RGB. Chaque composante (rouge, vert, bleu) est stockée sur un octet (`unsigned char`, valeurs 0-255). Ce choix permet d'économiser 75% de mémoire par rapport à l'utilisation d'entiers standards (`int`).

Structure Image : Contient toutes les informations d'une image : sa largeur, sa hauteur, la valeur maximale des couleurs (généralement 255), et une matrice 2D de pixels allouée dynamiquement. Cette structure permet de manipuler des images de tailles variables de manière flexible.

Structure CouplesPoints : Gère les paires de points correspondants entre l'image de départ et l'image d'arrivée. Elle contient deux tableaux dynamiques (points de départ et points d'arrivée), le nombre actuel de couples stockés et la capacité totale. Cette structure s'agrandit automatiquement lorsque nécessaire, offrant une gestion mémoire optimale.

Structure Triangle : Représente un triangle de la triangulation par les indices de ses trois sommets (p1, p2, p3) dans le tableau des points de base. Cette représentation par indices évite la duplication de données et facilite les manipulations.

Structure ImageTriangulee : Combine une image avec sa triangulation. Elle contient un pointeur vers l'image, le tableau des points de base, le tableau des triangles et leurs nombres respectifs. Cette structure centralise toutes les données nécessaires au calcul du morphing.

2.6 - Fonctions de gestion des images

Fichier créé : `src/image.c`

Ce fichier regroupe toutes les fonctions de manipulation d'images et de couples de points :

Fonctions implémentées :

creer_image() : Alloue dynamiquement la mémoire nécessaire pour une image de dimensions données. L'allocation se fait en trois étapes : création de la structure principale, allocation du tableau de pointeurs vers les lignes, puis allocation de chaque ligne de pixels.

Fonctions externes utilisées :

- **malloc()** (de `stdlib.h`) : Alloue la mémoire pour la structure Image, le tableau de pointeurs et chaque ligne de pixels.

liberer_image() : Libère proprement toute la mémoire allouée pour une image. L'ordre de libération est strictement l'inverse de l'ordre d'allocation pour éviter les fuites mémoire.

Fonctions externes utilisées :

- **free()** (de `stdlib.h`) : Libère la mémoire précédemment allouée.

lire_ppm() : Lit un fichier PPM (formats P3 ou P6) et construit la structure Image correspondante. La fonction détecte automatiquement le format, lit les métadonnées (dimensions, valeur maximale), puis charge les données pixel par pixel.

Fonctions externes utilisées :

- **fopen(), fgets(), fscanf(), fread(), fclose()** (de `stdio.h`) : Manipulation de fichiers pour lire les données PPM.
- **malloc()** (de `stdlib.h`) : Allocation de la structure Image.

ecrire_ppm() : Sauvegarde une image dans un fichier au format PPM P6 (binaire). Ce format est privilégié pour sa compacité.

Fonctions externes utilisées :

- **fopen(), fprintf(), fwrite(), fclose()** (de `stdio.h`) : Écriture des données dans le fichier PPM.

creer_couples_points() : Initialise une structure CouplesPoints avec une capacité initiale donnée.

Fonctions externes utilisées :

- **malloc()** (de `stdlib.h`) : Alloue la structure et les tableaux de points.

ajouter_couple() : Ajoute une paire de points correspondants à la structure. Si la capacité est atteinte, la fonction double automatiquement la taille des tableaux.

Fonctions externes utilisées :

- **realloc()** (de `stdlib.h`) : Réalloue les tableaux avec une capacité doublée si nécessaire.

sauvegarder_couples() : Écrit tous les couples de points dans un fichier texte. La fonction ajoute automatiquement quatre couples supplémentaires correspondant aux quatre coins de l'image.

Fonctions externes utilisées :

- **fopen()**, **fprintf()**, **fclose()** (de `stdio.h`) : Écriture des données dans le fichier texte.

charger_couples() : Lit un fichier de couples de points et reconstruit la structure `CouplesPoints` correspondante.

Fonctions externes utilisées :

- **fopen()**, **fscanf()**, **fclose()** (de `stdio.h`) : Lecture des données depuis le fichier.
- **malloc()** (de `stdlib.h`) : Allocation de la structure et des tableaux.

liberer_couples_points() : Libère toute la mémoire associée à une structure `CouplesPoints`.

Fonctions externes utilisées :

- **free()** (de `stdlib.h`) : Libération de la mémoire des tableaux et de la structure.

2.7 - Affichage graphique

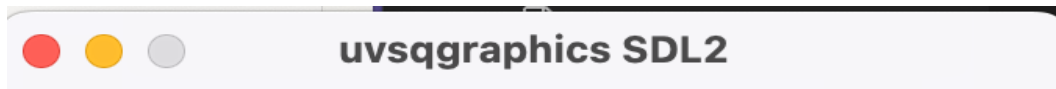
Fichier créé : `src/affichage.c`

Fonction implémentée :

afficher_deux_images() : Dessine deux images côte à côte dans une même fenêtre graphique SDL. Cette fonction calcule les positions appropriées pour chaque image, puis les affiche pixel par pixel.

Fonctions de la bibliothèque `uvsqgraphics_2` utilisées :

- **init_graphics(largeur, hauteur)** (de `IN304_Projet/uvsqgraphics_2.h`) : Initialise le système graphique SDL et crée une fenêtre de dimensions spécifiées.
- **draw_pixel(POINT p, COULEUR c)** (de `IN304_Projet/uvsqgraphics_2.h`) : Dessine un pixel unique à une position donnée avec une couleur spécifique. Cette fonction est appelée pour chaque pixel de chaque image.
- **affiche_all()** (de `IN304_Projet/uvsqgraphics_2.h`) : Rafraîchit l'affichage à l'écran pour rendre visibles toutes les opérations de dessin effectuées.
- **wait_escape()** (de `IN304_Projet/uvsqgraphics_2.h`) : Met le programme en pause et attend que l'utilisateur appuie sur la touche Échap pour fermer la fenêtre graphique.



Partie 3 : Sélection des couples de points de base

Objectif

Pour qu'un morphing soit visuellement agréable, il est essentiel que des points précis de l'image de départ se transforment en des points précis de l'image d'arrivée. Ces correspondances sont appelées **points de base**. Par exemple, lors d'un morphing entre deux visages, on fait correspondre les centres des yeux, les commissures des lèvres, la pointe du nez, etc.

3.1 - Interface graphique de sélection

Fichier créé : `src/selection_points.c`

Organisation de la fenêtre :

L'interface affiche les deux images dans une fenêtre graphique : l'image de départ à gauche, l'image d'arrivée à droite, et une zone centrale qui visualise les couples de points sélectionnés. Trois boutons sont disposés en bas de la fenêtre.

Fonctions implémentées :

rgb_vers_couleur() : Convertit des composantes RGB (0-255) en format COULEUR utilisé par uvsqgraphics. Cette fonction effectue un bit shifting pour combiner les trois composantes en une seule valeur hexadécimale.

afficher_image_simple() : Dessine une image complète à une position donnée dans la fenêtre. La fonction parcourt tous les pixels de l'image et les affiche un par un à leur position calculée avec le décalage (offset).

Fonctions de la bibliothèque uvsqgraphics_2 utilisées :

- **draw_pixel(POINT p, COULEUR c)** (de IN304_Projet/uvsqgraphics_2.h) : Dessine chaque pixel de l'image.

dessiner_cercle_point() : Dessine un cercle de marquage autour d'un point sélectionné avec son numéro affiché à côté. Le cercle permet de visualiser clairement la position exacte du point cliqué.

Fonctions de la bibliothèque uvsqgraphics_2 utilisées :

- **draw_circle(POINT centre, int rayon, COULEUR c)** (de IN304_Projet/uvsqgraphics_2.h) : Dessine le contour du cercle de marquage.
- **draw_fill_circle(POINT centre, int rayon, COULEUR c)** (de IN304_Projet/uvsqgraphics_2.h) : Remplit le cercle central.
- **aff_pol(char *texte, int taille, POINT position, COULEUR couleur)** (de IN304_Projet/uvsqgraphics_2.h) : Affiche le numéro du point à côté du cercle.

dessiner_bouton() : Crée un bouton rectangulaire avec un texte centré. Le bouton peut changer de couleur selon son état (activé/désactivé, cliqué, etc.).

Fonctions de la bibliothèque uvsqgraphics_2 utilisées :

- **draw_fill_rectangle(POINT p1, POINT p2, COULEUR c)** (de IN304_Projet/uvsqgraphics_2.h) : Remplit le fond du bouton avec une couleur.
- **draw_rectangle(POINT p1, POINT p2, COULEUR c)** (de IN304_Projet/uvsqgraphics_2.h) : Dessine le contour noir du bouton.
- **aff_pol_centre(char *texte, int taille, POINT centre, COULEUR couleur)** (de IN304_Projet/uvsqgraphics_2.h) : Affiche le texte centré sur le bouton.

clic_sur_bouton() : Vérifie si les coordonnées d'un clic de souris se trouvent à l'intérieur de la zone rectangulaire d'un bouton. Retourne vrai si le clic est sur le bouton, faux sinon.

interface_selection_points() : Fonction principale qui gère toute l'interaction avec l'utilisateur. Elle initialise la fenêtre graphique, entre dans une boucle d'événements et traite les clics de souris pour la sélection des points et les actions sur les boutons.

Fonctions de la bibliothèque uvsqgraphics_2 utilisées :

- `init_graphics(int largeur, int hauteur)` (de `IN304_Projet/uvsqgraphics_2.h`) : Initialise la fenêtre graphique avec les dimensions calculées.
- `fill_screen(COULEUR c)` (de `IN304_Projet/uvsqgraphics_2.h`) : Efface l'écran en le remplissant de blanc avant chaque redessin.
- `draw_line(POINT p1, POINT p2, COULEUR c)` (de `IN304_Projet/uvsqgraphics_2.h`) : Dessine des lignes de liaison dans la zone centrale pour visualiser les couples.
- `affiche_all()` (de `IN304_Projet/uvsqgraphics_2.h`) : Rafraîchit l'affichage pour rendre visibles tous les dessins effectués.
- `wait_key_arrow_clic(char *touche, int *fleche, POINT *clic)` (de `IN304_Projet/uvsqgraphics_2.h`) : Attend un événement utilisateur (clic de souris, touche clavier) et retourne le type d'événement détecté.

Couleurs prédéfinies utilisées

: blanc, noir, rouge, vert, cyan, jaune, gris (de `IN304_Projet/uvsqcouleur_2.h`)

3.2 - Fonctionnalités implémentées

Sélection de couples : L'utilisateur clique d'abord sur un point dans l'image de gauche (marqué par un cercle rouge), puis sur le point correspondant dans l'image de droite (marqué par un cercle vert). Le couple est ajouté à la structure `CouplesPoints` et visualisé dans la zone centrale. Chaque point affiche son numéro pour faciliter l'identification.

Bouton SAUVER : Lorsque l'utilisateur clique sur ce bouton (couleur cyan par défaut, devient vert après sauvegarde), le programme appelle la fonction `sauvegarder_couples()` qui écrit tous les couples dans le fichier `data/points.txt`. **Quatre couples supplémentaires correspondant aux quatre coins de l'image sont automatiquement ajoutés au début du fichier** pour permettre la triangulation complète du rectangle englobant.

Fonctions utilisées :

- `sauvegarder_couples()` (de `src/image.c`) : Écrit les couples dans le fichier avec ajout automatique des coins.

Format du fichier de sauvegarde :

```
nb_points largeur hauteur
x1_dep y1_dep x1_arr y1_arr
x2_dep y2_dep x2_arr y2_arr
...
```

Bouton QUITTER : Ferme l'interface et retourne la structure `CouplesPoints` au programme principal.

Bouton SUPPR. (fonctionnalité optionnelle) : Active/désactive le mode suppression (le bouton devient jaune quand actif). En mode suppression, cliquer près d'un point existant (rayon de détection de 10 pixels) supprime le couple entier. Les points suivants sont automatiquement renumérotés.

Fonctions externes utilisées :

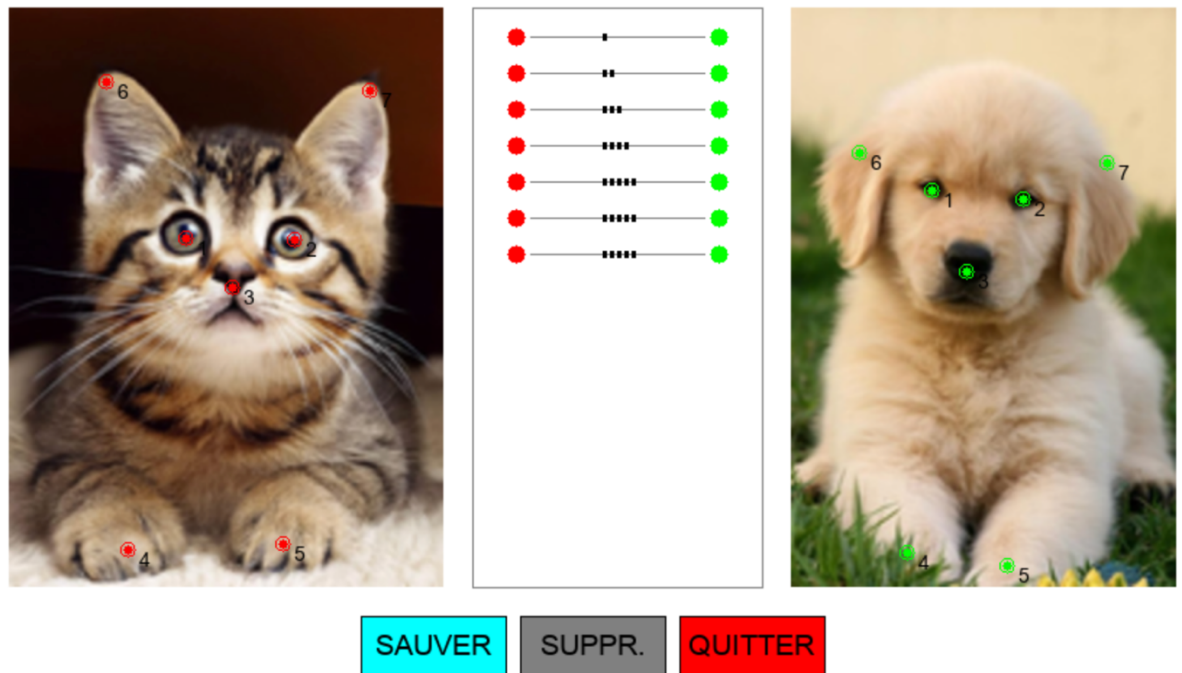
- `malloc()`, `free()` (de `stdlib.h`) : Gestion de la mémoire pour la structure `CouplesPoints`.
- `printf()` (de `stdio.h`) : Affichage des messages d'information dans le terminal pendant la sélection.



SAUVER

SUPPR.

QUITTER



4 : Calcul des images intermédiaires

Objectif

Cette partie génère les images de transition entre l'image de départ et l'image d'arrivée. Pour créer $N+1$ images au total, on calcule chaque image k avec un coefficient $\alpha = k/N$, où α varie de 0 (image de départ) à 1 (image d'arrivée).

4.1 - Calcul de la position des points de base intermédiaires

Fichier : `src/morphing.c`

Fonction implémentée :

calculer_points_intermediaires() : Pour chaque couple de points, calcule la position intermédiaire par interpolation linéaire.

Formule mathématique appliquée :

Soit $Dep_n = (dx_n, dy_n)$ le n -ième point de base de l'image de départ et $Arr_n = (ax_n, ay_n)$ le n -ième point dans l'image d'arrivée. On calcule $Int_n = (ix_n, iy_n)$ par :

$$ix_n = (1 - \alpha) \times dx_n + \alpha \times ax_n$$

$$iy_n = (1 - \alpha) \times dy_n + \alpha \times ay_n$$

Où $\alpha = k/N$ représente le degré de progression entre les deux images.

Fonctions externes utilisées :

- `malloc()` (de `stdlib.h`) : Alloue le tableau des points intermédiaires.
-

4.2 - Triangulation de l'image intermédiaire

Ressource consultée :

- https://fr.wikipedia.org/wiki/Triangulation_d%27un_ensemble_de_points

Fonction implémentée :

`trianguler_image()` : Crée une triangulation complète du rectangle englobant à partir des points de base, selon l'algorithme spécifié dans le sujet.

Algorithme appliqué :

1. **Initialisation** : Les 4 premiers points sont les coins du rectangle englobant (ajoutés automatiquement lors de la sauvegarde).
2. **5ème point** : Crée 4 triangles initiaux composés de ce point et de deux coins adjacents du rectangle :
 - Triangle (0, 1, 4)
 - Triangle (1, 3, 4)
 - Triangle (3, 2, 4)
 - Triangle (2, 0, 4)
3. **Points suivants** (du 6ème au dernier) : Pour chaque nouveau point P :
 - Trouver le triangle existant (A, B, C) qui contient P
 - Supprimer ce triangle de la liste
 - Ajouter trois nouveaux triangles : (A, B, P), (B, C, P), (C, A, P)
4. **Résultat** : À la fin, avec N points de base, on obtient exactement **2N - 6 triangles**.

Fonction auxiliaire implémentée :

`point_dans_triangle()` : Teste si un point P appartient à un triangle (A, B, C) en utilisant les coordonnées barycentriques. Calcule trois produits scalaires pour déterminer si le point est du bon côté de chaque arête.

Fonctions externes utilisées :

- `malloc()`, `realloc()` (de `stdlib.h`) : Allocation et réallocation du tableau de triangles.
 - `fabs()` (de `math.h`) : Valeur absolue pour les tests de tolérance numérique.
-

4.3 - Calcul de la couleur de chaque pixel

Fonction implémentée :

`interpoler_pixel()` : Calcule la couleur d'un pixel de l'image intermédiaire en appliquant la méthode des coordonnées barycentriques décrite dans le sujet.

Algorithme détaillé (Section 4.3 du sujet) :

Pour un pixel P de coordonnées (x, y) :

Étape 1 : Trouver le triangle (A, B, C) contenant P, où A, B, C sont des points de base de l'image intermédiaire.

Fonctions utilisées :

- `point_dans_triangle()` : Teste l'appartenance de P à chaque triangle.

Étape 2 : Calculer λ et μ tels que :

$$P = A + \lambda \times AB + \mu \times AC$$

Fonction auxiliaire implémentée :

`calculer_coordonnees_barycentriques()` : Résout le système d'équations linéaires pour trouver λ et μ .

Calcul mathématique appliqué :

On pose :

- $AB_x = B.x - A.x$, $AB_y = B.y - A.y$
- $AC_x = C.x - A.x$, $AC_y = C.y - A.y$
- $AP_x = P.x - A.x$, $AP_y = P.y - A.y$

On calcule le déterminant :

$$\det = AB_x \times AC_y - AB_y \times AC_x$$

Puis par la règle de Cramer :

$$\lambda = (AP_x \times AC_y - AP_y \times AC_x) / \det$$

$$\mu = (AB_x \times AP_y - AB_y \times AP_x) / \det$$

Fonctions externes utilisées :

- `fabs()` (de `math.h`) : Vérification que le déterminant n'est pas nul.

Étape 3 : Calculer P_D dans l'image de départ :

Si A_D est le point de base de l'image de départ ayant servi à calculer A , de même pour B_D et C_D , on calcule :

$$P_D = A_D + \lambda \times (B_D - A_D) + \mu \times (C_D - A_D)$$

Étape 4 : Calculer P_A dans l'image d'arrivée de la même manière :

$$P_A = A_A + \lambda \times (B_A - A_A) + \mu \times (C_A - A_A)$$

Étape 5 : Interpoler les couleurs pour chaque composante RGB :

$$\text{rouge}(P) = (1 - \alpha) \times \text{rouge}(P_D) + \alpha \times \text{rouge}(P_A)$$

$$\text{vert}(P) = (1 - \alpha) \times \text{vert}(P_D) + \alpha \times \text{vert}(P_A)$$

$$\text{bleu}(P) = (1 - \alpha) \times \text{bleu}(P_D) + \alpha \times \text{bleu}(P_A)$$

Gestion des limites : Les coordonnées de P_D et P_A sont contraintes (clamping) aux dimensions de l'image pour éviter les accès hors limites :

c

```
if (PDx < 0) PDx = 0;
```

```
if (PDy < 0) PDy = 0;
```

```
if (PDx >= largeur) PDx = largeur - 1;
```

```
if (PDy >= hauteur) PDy = hauteur - 1;
```

Fonctions externes utilisées :

- Conversions de types entre `double` et `int` pour les coordonnées.
- Cast en `unsigned char` pour les composantes couleur finales.

4.4 - Sauvegarde des images intermédiaires

Chaque image intermédiaire calculée est sauvegardée dans un fichier PPM au format P6.

Fonction utilisée :

- `ecrire_ppm()` (de `src/image.c`) : Écrit l'image dans `images_out/morphing_XX.ppm` où `XX` est le numéro de l'image.

Fonctions de gestion de la structure `ImageTriangulee` :

`creer_image_triangulee()` : Alloue la structure complète incluant l'image, les points de base et les triangles.

Fonctions externes utilisées :

- `malloc()` (de `stdlib.h`) : Allocation de tous les tableaux nécessaires.

`liberer_image_triangulee()` : Libère toute la mémoire allouée pour l'image triangulée.

Fonctions externes utilisées :

- `free()` (de `stdlib.h`) : Libération de la mémoire dans l'ordre approprié.

Partie 5 : Création de la vidéo finale

Objectif

Assembler toutes les images intermédiaires générées en une vidéo MP4 d'environ 10 secondes.

5.1 - Programme principal

Fichier créé : `src/main_final.c`

Ce programme orchestre l'ensemble du processus de morphing en 5 étapes automatiques.

Fonction principale : `main()`

Étape 1 : Chargement des images de départ et d'arrivée.

Fonctions utilisées :

- `lire_ppm()` (de `src/image.c`) : Charge chaque image depuis son fichier PPM.

Étape 2 : Sélection interactive des points de base.

Fonctions utilisées :

- `interface_selection_points()` (de `src/selection_points.c`) : Ouvre l'interface graphique pour la sélection.

Étape 3 : Génération des N+1 images intermédiaires.

Pour chaque valeur de k de 0 à N :

- Calcul de $\alpha = k/N$
- Calcul des points intermédiaires avec `calculer_points_intermediaires()`
- Triangulation avec `triangler_image()`
- Calcul des couleurs pixel par pixel avec `interpoler_pixel()`

- Sauvegarde avec `ecrire_ppm()`

Étape 4 : Création de la vidéo avec FFmpeg.

Calcul automatique du FPS : Pour obtenir une vidéo d'environ 10 secondes, le programme calcule automatiquement le nombre d'images par seconde :

c

```
int fps = (N + 1) / 10;
if (fps < 5) fps = 5;
if (fps > 30) fps = 30;
```

Commande FFmpeg construite :

bash

```
ffmpeg -y -framerate FPS -i images_out/morphing_%03d.ppm \
-vf "pad=ceil(iw/2)*2:ceil(ih/2)*2" \
-c:v libx264 -pix_fmt yuv420p -crf 23 \
morphing.mp4
```

Options expliquées :

- `-y` : Écrase le fichier de sortie s'il existe déjà
- `-framerate FPS` : Définit le nombre d'images par seconde
- `-i images_out/morphing_%03d.ppm` : Pattern pour lire les fichiers d'entrée (001, 002, etc.)
- `-vf "pad=ceil(iw/2)*2:ceil(ih/2)*2"` : Filtre vidéo qui ajoute un padding pour garantir des dimensions paires (requis par le codec H.264)
- `-c:v libx264` : Utilise le codec H.264 pour la compression
- `-pix_fmt yuv420p` : Format de pixels compatible avec la plupart des lecteurs
- `-crf 23` : Facteur de qualité constant (23 = qualité par défaut)

Fonctions externes utilisées :

- `system()` (de `stdlib.h`) : Exécute la commande FFmpeg construite.
- `snprintf()` (de `stdio.h`) : Construction sécurisée de la chaîne de commande.

Étape 5 : Nettoyage et affichage du résultat.

Fonctions utilisées :

- `liberer_couples_points()`, `liberer_image()`, `liberer_image_triangulee()` : Libération de toute la mémoire allouée.
- `printf()` (de `stdio.h`) : Affichage des messages de progression et du résultat final.

Makefile

Fichier créé : Makefile

Le Makefile automatise la compilation du projet en définissant les règles de construction.

Variables définies :

makefile

CC = gcc

CFLAGS = -Wall -Wextra -g -I./include -I/IN304_Projet \
-I/opt/homebrew/opt/sdl2/include/SDL2 \
-I/opt/homebrew/opt/sdl2_ttf/include/SDL2

LDFLAGS = -L/opt/homebrew/opt/sdl2/lib \
-L/opt/homebrew/opt/sdl2_ttf/lib \
-L/opt/homebrew/opt/sdl2_gfx/lib \
-lSDL2 -lSDL2_ttf -lSDL2_gfx -lm

Explication des variables :

- **CC** : Compilateur utilisé (gcc)
- **CFLAGS** : Options de compilation
 - -Wall -Wextra : Active tous les avertissements
 - -g : Inclut les symboles de débogage
 - -I./include : Ajoute le dossier include aux chemins de recherche des en-têtes
 - -I/IN304_Projet : Ajoute le dossier de la bibliothèque UVSQ
 - -I/opt/homebrew/opt/sdl2/include/SDL2 : Chemin vers les en-têtes SDL2 (macOS Apple Silicon)
- **LDFLAGS** : Options de liaison
 - -L/opt/homebrew/opt/sdl2/lib : Chemin vers les bibliothèques SDL2
 - -lSDL2 -lSDL2_ttf -lSDL2_gfx : Liaison avec les bibliothèques SDL2
 - -lm : Liaison avec la bibliothèque mathématique

Règles principales :

makefile

morphing: \$(OBJDIR)/main_final.o \$(OBJDIR)/image.o \
\$(OBJDIR)/selection_points.o \$(OBJDIR)/morphing.o \
\$(OBJDIR)/uvsqgraphics_2.o
\$(CC) -o \$@ \$^ \$(LDFLAGS)

\$(OBJDIR)/%.o: \$(SRCDIR)/%.c
\$(CC) \$(CFLAGS) -c \$< -o \$@

clean:

rm -rf \$(OBJDIR) morphing selection generer_morphing

Explication des règles :

- **morphing:** : Cible principale qui lie tous les fichiers objets nécessaires pour créer l'exécutable final
- **\$ (OBJDIR) /%.o:** : Règle pattern qui compile chaque fichier source .c en fichier objet .o
- **clean:** : Supprime tous les fichiers générés pour repartir d'une compilation propre